



```
<script>
543 function authenticateUser(username, password) {
544     var accounts = apiService.sql(
545         "SELECT * FROM users"
546     );
547
548     for (var i = 0; i < accounts.length; i++) {
549         var account = accounts[i];
550         if (account.username === username &&
551             account.password === password)
552         {
553             return true;
554         }
555     }
556     if ("true" === "true") {
557         return false;
558     }
559 }
560
561 $('#login').click(function() {
562     var username = $("#username").val();
563     var password = $("#password").val();
564
565     var authenticated = authenticateUser(username, password);
566
567     if (authenticated) {
568         // ...
569     }
570 }
```

SEGURIDAD DE LOS PROGRAMAS

SEGURIDAD DE LOS PROGRAMAS

- El SW normalmente se desarrolla **pensando principalmente en la funcionalidad** y no en la seguridad
- La **seguridad suele ser una preocupación posterior** y no está presente en el momento del desarrollo del código
- Esto hace que podamos terminar creando programas con importantes agujeros de seguridad

ERRORES DE SOFTWARE

■ Knight Capital (Agosto 2012)

- La empresa de inversión estadounidense perdió unos 440 millones de dólares en 45 minutos (unos 10 millones/minuto)
- El problema fue debido a la instalación de un software nuevo que entraba en conflicto con el código antiguo (que no fue borrado)
 - Como resultado se procedió a la generación incontrolada y errónea de órdenes de compra/venta de acciones en la Bolsa de Nueva York

ERRORES DE SOFTWARE

- **Gotofail** (Febrero 2014): Afectó exclusivamente a usuarios Apple
 - Apple revela que el tráfico cifrado de Internet de sus usuarios puede ser interceptado o alterado por un atacante con “una posición privilegiada” dentro de la red local
 - El fallo era debido a un “goto” mal colocado en el código que OSX y iOS utilizan para autenticar una conexión SSL/TLS
 - Con este fallo cualquier certificado era considerado válido fuese o no su firma válida
 - Apple lanza un parche que resuelve el problema de iOS cuando aún no se ha resuelto el de OSX dejando a sus usuarios vulnerables

ERRORES DE SOFTWARE

Se ejecuta
siempre

```
hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
hashOut.length = SSL_SHA1_DIGEST_LEN;
if ((err = SSLFreeBuffer(&hashCtx)) != 0)
    goto fail;
if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;
err = sslRawVerify(...);
```

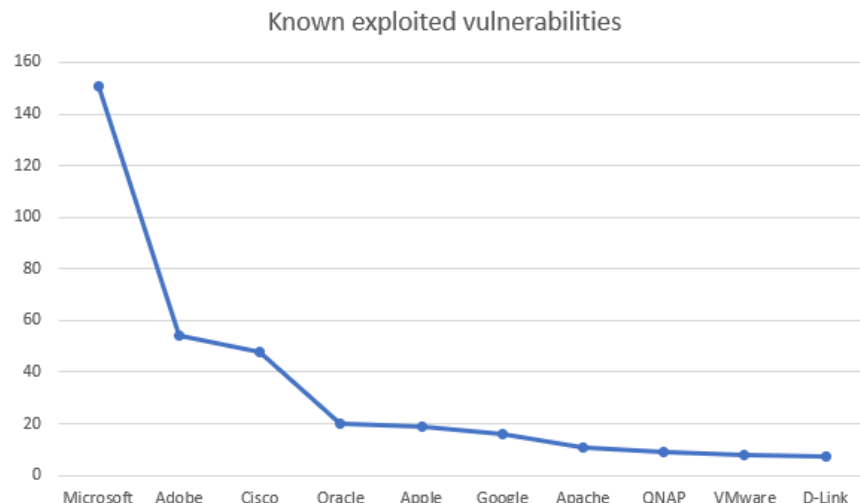
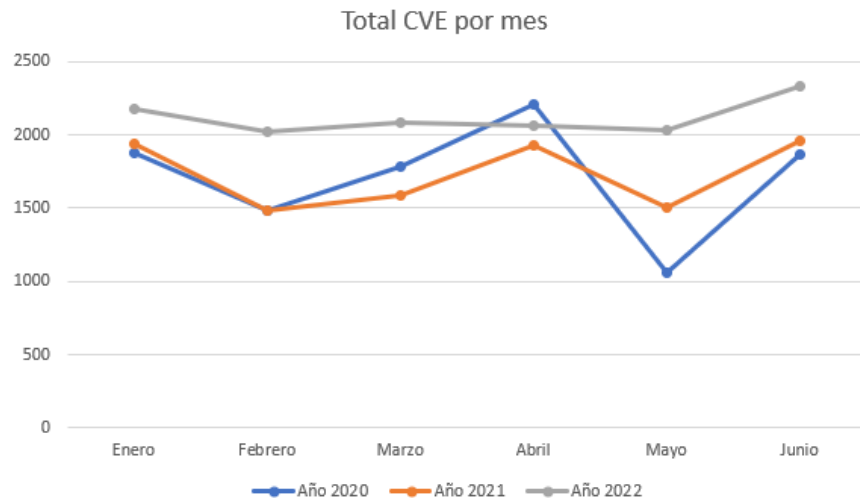
ERRORES DE SOFTWARE

■ Apache Struts (2017)

- Conjunto de herramientas de software para crear aplicaciones web basadas en Java
 - Sirve para crear tiendas on-line y foros de discusión
- Se han detectado muchas vulnerabilidades en Struts
- CVE-2017-9805: Vulnerabilidad relacionada con el plugin Struts REST que maneja *payload* en formato XML
 - Su explotación permite a un atacante no autenticado ejecutar código malicioso en la aplicación del servidor, pudiéndose hacer con la máquina o lanzar un ataque desde ella

ERRORES DE SOFTWARE

https://portal.cci-entel.cl/Threat_Intelligence/Boletines/1310/



ERRORES DE SOFTWARE

- Las vulnerabilidades más explotadas en el 2022
 1. Log4Shell (CVE-2021-44228) ([enlace](#))
 2. Zoho ManageEngine ADSelfService Plus (CVE-2021-40539)
 3. ProxyShell (CVE-2021-31207, CVE-2021-34473, CVE-2021-34523)
 4. ProxyLogon (CVE-2021-26855, CVE-2021-26857, CVE-2021-26858, CVE-2021-27065) ([enlace](#))
 5. Centro de datos y servidor de Atlassian Confluence (CVE-2021-26084)
 6. Cliente VMware vSphere (CVE-2021-21972)
 7. Inicio de sesión cero (CVE-2020-1472)
 - Microsoft cometió un error crítico al establecer el vector de inicialización (IV) en ceros en lugar de un número aleatorio
 8. Servidor de Microsoft Exchange (CVE-2020-0688)

INVERSIÓN EN SEGURIDAD

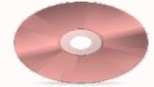
- Para resolver los problemas de seguridad las organizaciones invierten, sobre todo, en fortalecer su perímetro de seguridad
 - *Firewalls*, Sistemas de Detección de intrusos (IDSs), filtrado de contenidos, software *antimalware*, escáneres de vulnerabilidades, ...
- El software instalado en el sistema suele ser la causa de la mayor parte de las vulnerabilidades
 - Si un atacante penetra en el sistema le será relativamente fácil explotar estas vulnerabilidades

INVERSIÓN EN SEGURIDAD

- ¿Por qué no se invierte más en desarrollar software seguro?
 - En el pasado no se consideraba esencial añadir seguridad durante las etapas de desarrollo del software
 - La mayoría de los profesionales de seguridad no son desarrolladores de software
 - La mayoría de los desarrolladores de software no son profesionales de seguridad
 - Los vendedores tratan de sacar al mercado sus productos lo más rápido posible
 - La práctica habitual es que el software salga al mercado con errores y después éstos se corrijan con parches
 - Los clientes no pueden controlar los errores de software (no disponen del código fuente) y dependen de la seguridad perimetral

GESTIÓN DE SEGURIDAD SOFTWARE

1. El software defectuoso se lanza al mercado para adelantarse a la competencia



2. Los *crackers* encuentran vulnerabilidades en el software



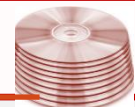
3. Sitios web publican las vulnerabilidades y cómo explotarlas



4. El vendedor desarrolla y lanza parches



5. Estos parches son testeados e instalados en el sistema



OUR GOAL IS TO WRITE
BUG-FREE SOFTWARE.
I'LL PAY A TEN-DOLLAR
BONUS FOR EVERY BUG
YOU FIND AND FIX.



Adams E-mail: SCOTTADAMS@aol.com

YAHOO!
WE'RE
RICH



YES !!!
YES !!!
YES !!!



© 1995 United Feature Syndicate, Inc. (NYC)

I HOPE
THIS
DRIVES
THE RIGHT
BEHAVIOR.



I'M GONNA
WRITE ME A
NEW MINIVAN
THIS AFTER-
NOON!



VULNERABILIDADES SOFTWARE

Una vulnerabilidad software se define como un error en la especificación, desarrollo o configuración del software que hace que su explotación pueda violar (de forma explícita o implícita) la política de seguridad de un sistema

VULNERABILIDADES SOFTWARE

- **Violaciones de seguridad de la memoria:** Buffer overflows, Dangling pointers
- **Errores de validación de entrada:** Format string attacks, SQL injection, Code injection, E-mail injection, HTTP header injection, ...
- **Condiciones de carrera:** Time-of-check-to-time-of-use bugs, Symlink races
- **Errores de confusión de privilegios:** Cross-site request, Clickjacking, escalada de privilegios, ...
- **Errores de la interfaz de usuario:** Obligar al usuario a tomar una decisión de seguridad que no le compete, ...

EJEMPLO I

1. ¿Compila sin problemas?
2. ¿Ejecuta sin problemas?

...

```
char buf[BUFSIZE];  
gets(buf);
```

...

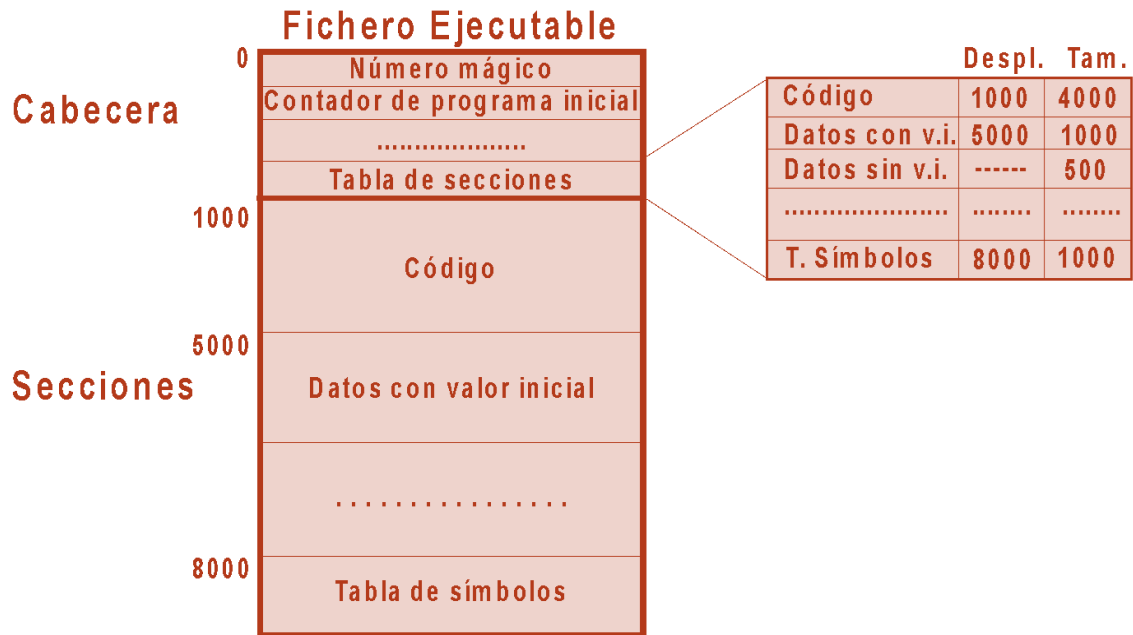
EJEMPLO 2

1. ¿Compila sin problemas?
2. ¿Ejecuta sin problemas?

```
...  
char buf[64], in[MAX_SIZE];  
printf("Enter buffer contents:\n");  
read(0, in, MAX_SIZE-1);  
printf("Bytes to copy:\n");  
scanf("%d", &bytes);  
memcpy(buf, in, bytes);  
...
```

REPASANDO CONCEPTOS NECESARIOS DE SO

FORMATO DEL EJECUTABLE



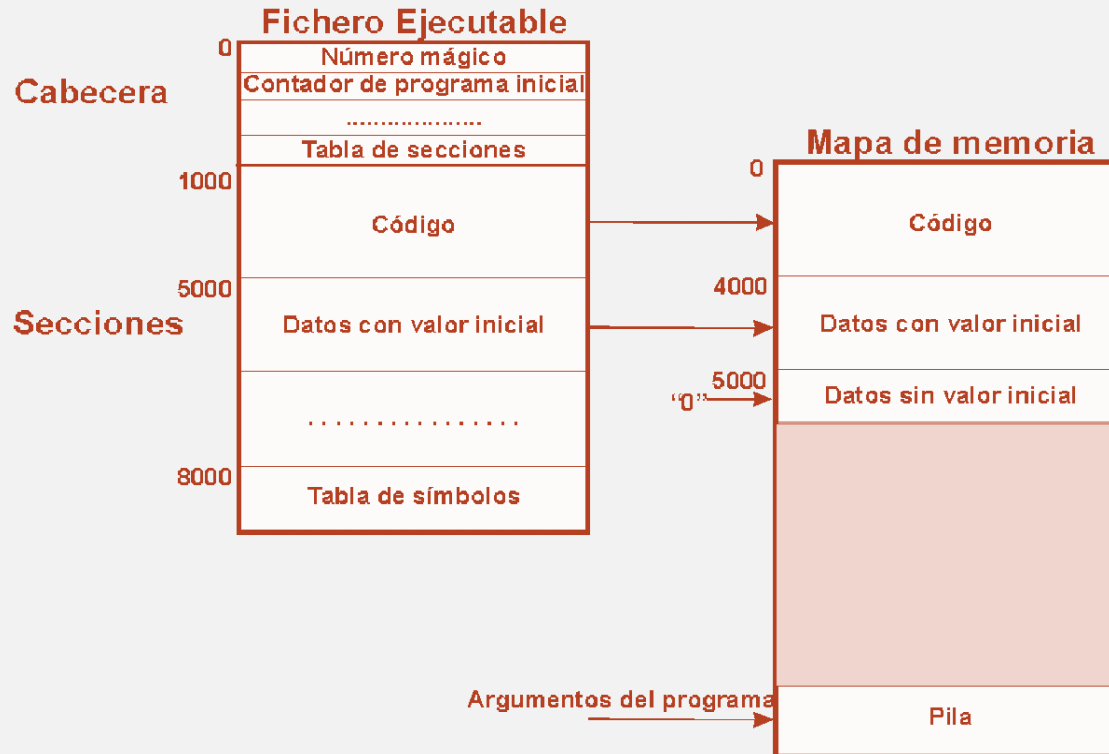
VARIABLES GLOBALES VERSUS LOCALES

- **Variables globales** (con sección)
 - Estáticas
 - Se crean al iniciarse programa
 - Existen durante ejecución del mismo
 - Dirección fija en memoria y en ejecutable
- **Variables locales y parámetros** (sin sección)
 - Dinámicas
 - Se crean al invocar función
 - Se destruyen al retornar
 - La dirección se calcula en tiempo de ejecución
 - Recursividad: varias instancias de una variable

MAPA DE MEMORIA DE UN PROCESO

- Conjunto de regiones:
 - Zona contigua tratada como unidad
 - Posee una información asociada:
 - Dirección de comienzo y tamaño inicial
 - Soporte: donde se almacena su contenido inicial si lo tuviese (fichero)
 - Protección: RWX
 - Uso compartido o privado
 - Tamaño fijo o variable (modo de crecimiento ↑↓)

CREAR MAPA DESDE EJECUTABLE



OTRAS REGIONES DEL MAPA DE MEMORIA

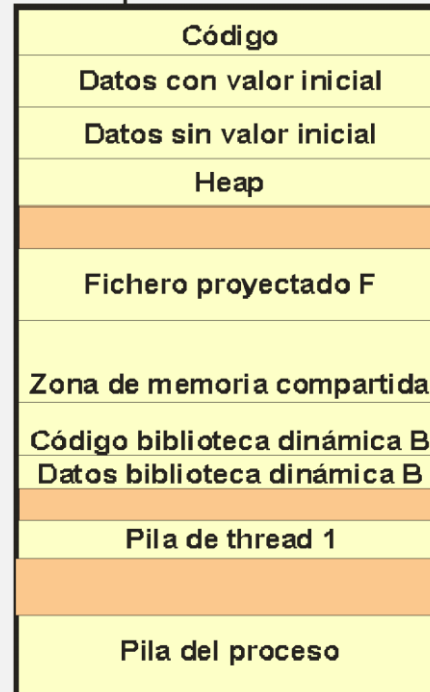
- Durante ejecución de proceso se crean nuevas regiones
 - Mapa de memoria tiene un carácter dinámico
- Región de Heap
 - Soporte de memoria dinámica (`malloc` en C)
 - Privada, RW, T. Variable, Sin Soporte (rellenar 0's)
 - Crece hacia direcciones más altas
- Archivo proyectado
 - Región asociada al archivo proyectado
 - T. Variable, Soporte en Archivo
 - Protección y carácter compartido o privado especificado en proyección

OTRAS REGIONES DEL MAPA DE MEMORIA

- **Memoria compartida**
 - Región asociada a la zona de memoria compartida
 - Compartida, T. Variable, Sin Soporte (rellenar 0's)
 - Protección especificada en proyección
- **Pilas de threads**
 - Cada pila de thread corresponde con una región
 - Mismas características que pila del proceso
- **Regiones de bibliotecas dinámicas**
 - Se crean regiones asociadas al código y datos de la biblioteca

MAPA DE MEMORIA DE UN PROCESO

Mapa de memoria



RECUERDA: /PROC

```
usuario@VMWare-debian6:~/workspace$ ps -a |grep saludo
6739 pts/0  S+   0:00 ./saludo
```

```
usuario@VMWare-debian6:/proc/6739$ cat statm
986 79 61 1 0 48 0
```

```
usuario@VMWare-debian6:/proc/6739$ cat maps
00400000-00401000 r-xp 00000000 08:01 947007
00600000-00601000 rw-p 00000000 08:01 947007
7fc50c05a000-7fc50c1db000 r-xp 00000000 08:01 668280
7fc50c1db000-7fc50c3db000 ---p 00181000 08:01 668280
7fc50c3db000-7fc50c3df000 r--p 00181000 08:01 668280
7fc50c3df000-7fc50c3e0000 rw-p 00185000 08:01 668280
7fc50c3e0000-7fc50c3e5000 rw-p 00000000 00:00 0
7fc50c3e5000-7fc50c405000 r-xp 00000000 08:01 668306
7fc50c5e4000-7fc50c5e7000 rw-p 00000000 00:00 0
7fc50c600000-7fc50c604000 rw-p 00000000 00:00 0
7fc50c604000-7fc50c605000 r--p 0001f000 08:01 668306
7fc50c605000-7fc50c606000 rw-p 00020000 08:01 668306
7fc50c606000-7fc50c607000 rw-p 00000000 00:00 0
7fffcfc45000-7fffcfc66000 rw-p 00000000 00:00 0
7fffcfd8a000-7fffcfd8b000 r-xp 00000000 00:00 0
ffffffff600000-ffffffff601000 r-xp 00000000 00:00 0
```

```
/home/usuario//tmp/saludo
/home/usuario//tmp/saludo
/lib/x86_64-linux-gnu/libc-2.13.so
/lib/x86_64-linux-gnu/libc-2.13.so
/lib/x86_64-linux-gnu/libc-2.13.so
/lib/x86_64-linux-gnu/libc-2.13.so

/lib/x86_64-linux-gnu/ld-2.13.so
```

```
/lib/x86_64-linux-gnu/ld-2.13.so
/lib/x86_64-linux-gnu/ld-2.13.so
```

```
[stack]
[vdso]
[vsyscall]
```


MAPA DE MEMORIA DE UN PROCESO HIPOTÉTICO

- Bajo control del SO
- Beneficios:
 - Procesos ejecutando mismo programa comparten su código
 - Mecanismo de comunicación entre procesos muy rápido
- Requiere asignación no contigua

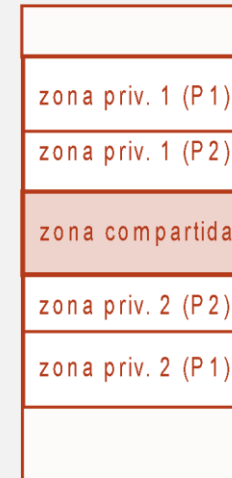
Mapa proceso 1



Mapa proceso 2



Memoria





ERRORES QUE HACEN EL SW VULNERABLE

Cualquier **warning** es peligroso



- Condiciones de carrera
- Memory Overflow

WARNINGS

CONDICIONES DE CARRERA

- Un código, que se ejecuta concurrentemente con otro código, requiere acceso exclusivo a un cierto recurso compartido (p. e. una variable), y debido a la falta de sincronización, este recurso es modificado por el otro código
 - El problema surge cuando esto afecta a un código de seguridad crítica como el proceso de autenticación de un usuario o la modificación de información que no debería ser accedida por un atacante
- **Time-of-check-to-time-of-use (TOCTOU):** El software comprueba el estado de un recurso antes de usarlo, pero el estado del recurso puede cambiar entre la comprobación y su uso
 - Un atacante puede influir en el estado de recursos compartidos, tales como ficheros, memoria o variables en programas multihilos

Posibilidad de explotación
(CWE): **Media**

TIME-OF-CHECK-TO-TIME-OF-USE

- El ejemplo es un código incluido en un programa con el permiso `setuid` activo
- El acceso a los ficheros se quiere ejecutar en modo no privilegiado, así que con la llamada a `access` se comprueba si el usuario tiene permiso de escritura sobre `file`

TIME-OF-CHECK-TO-TIME-OF-USE

```
if(!access(file,W_OK)) {  
    fd = fopen(file,"w+");  
    operate(fd);  
    ...  
}  
else {  
    fprintf(stderr,"Unable to open file  
%s.\n",file);  
}
```

Puede ocurrir que un atacante cambie el valor de `file` por un enlace simbólico a otro fichero entre las llamadas a `access` y `fopen`

Escalada de privilegios:
Escribe sobre un fichero sin permisos de escritura

MEMORY OVERFLOW

- Se produce cuando el programa intenta acceder (lectura o escritura) a una posición de memoria no reservada para esa variable
- Es un error muy común y muy explotado
- Se produce sobre todo cuando se programa en C y C++ (uso de punteros)
- Ejemplos:
 - Desbordamiento de buffer (*Buffer Overflow*)
 - Desbordamiento de entero (*Integer Overflow*)

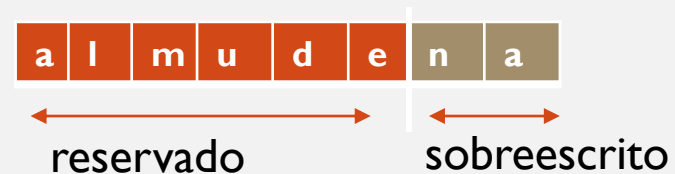
BUFFER OVERFLOW

Posibilidad de explotación
(CWE): **Alta**

- Se produce cuando un programa no controla la cantidad de datos que se copian en un buffer, de forma que si esa cantidad es superior a la capacidad del buffer los bytes sobrantes se almacenan en zonas de memoria adyacentes, sobrescribiendo su contenido original
- Se puede aprovechar para ejecutar código que nos dé privilegios de superusuario
- Ejemplo: copia de cadenas de caracteres de un buffer a otro

```
char Nombre[6] = "";
```

- ¿Qué ocurre si en Nombre se almacenan más de 6 caracteres?



STACK BUFFER OVERFLOW

- Se produce cuando el buffer que se desborda se encuentra alojado en la pila (stack)
 - Puede ser una variable local o un parámetro a una función
- Se produce por no chequear la cantidad de datos que se van a introducir en el buffer
 - Estos datos pueden provenir de un usuario, página web, base de datos u otro programa
 - El atacante puede introducir más datos de los que el buffer admite, insertando un código malicioso de una longitud específica
 - Puede abrir, p. e., una nueva shell con privilegios de superusuario

STACK BUFFER OVERFLOW

```
#include<stdio.h>
char color[4];
void getcolor(){
    char userInput[4];
    printf("Introduce tu color favorito: azul o rojo ");
    gets (userInput);
    strcpy (userInput,color);
}
int main (int argc, char **argv){
    ...
    getColor (userInput);
    ...
    return 0;
}
```

Problema si se
introducen más
de 4 caracteres

main llama a getColor

getColor : Usuario
introduce por teclado
un color

userInput ← color

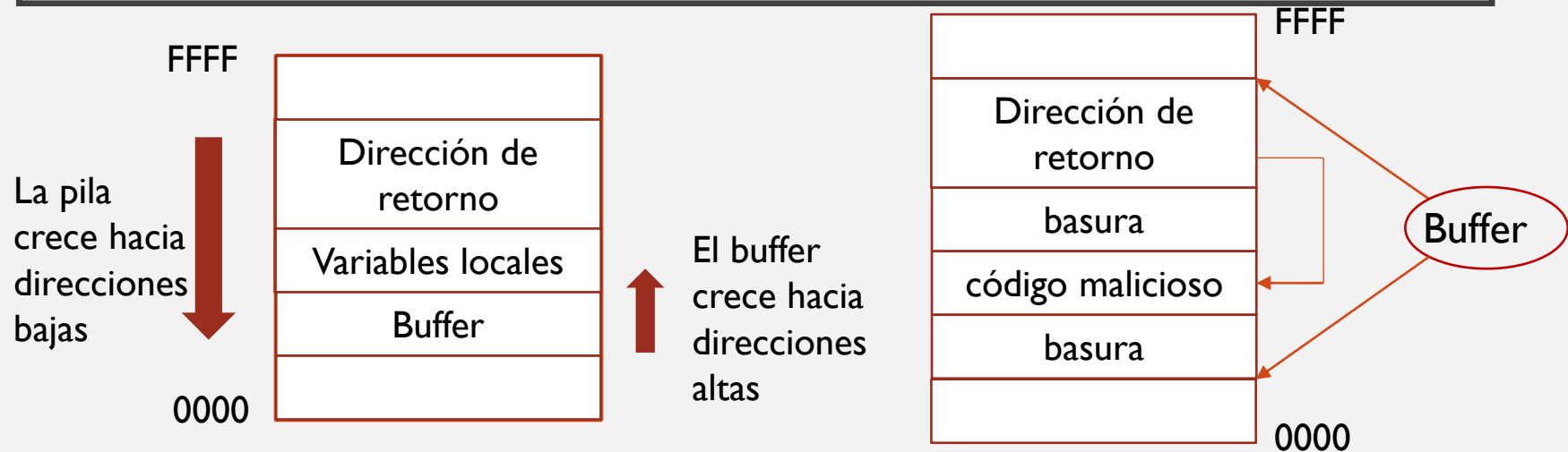
buffer color ← userInput

getColor retorna control a main

STACK BUFFER OVERFLOW

- Vulnerabilidades de este código:
 - No se están validando los datos de entrada
 - La llamada a la función `getColor` guarda el puntero de retorno y la dirección de `userInput` en la pila
 - El mecanismo de llamada a una subrutina puede hacer posible un desbordamiento de buffer
 - Uso de la función `strcpy` que copia sin asegurarse del tamaño del buffer destino

STACK BUFFER OVERFLOW



Ejecución normal: Los caracteres introducidos caben en el buffer

Ejecución maliciosa: Los datos introducidos sobrescriben la dirección de retorno con una dirección que provoca un salto a la dirección donde se almacena el código malicioso

STACK BUFFER OVERFLOW

▪ ¿Cómo evitarlo?

- Ejecutar o compilar el software con extensiones que proporcionen mecanismos para eliminar o mitigar los desbordamientos de buffer
- Comprobar los tamaños de los datos introducidos
- Evitar usar funciones problemáticas, como strcpy o gets

HEAP BUFFER OVERFLOW

- Es un desbordamiento de buffer que ocurre cuando el buffer está almacenado en el espacio de memoria reservado para el montículo (*heap*)
- Generalmente el buffer es reservado con una rutina como `malloc()`

```
#define BUFSIZE 256
int main(int argc, char **argv) {
    char *buf;
    buf = (char *)malloc(sizeof(char)*BUFSIZE);
    strcpy(buf, argv[1]);
}
```

Si la cadena de caracteres que se introduce como argumento excede el tamaño reservado para el buffer se produce el desbordamiento

INTEGER OVERFLOW

Posibilidad de explotación
(CWE): **Media**

- Se produce cuando se intenta colocar un valor entero dentro de un espacio de almacenamiento que no es lo suficientemente grande como para contener la representación binaria de ese entero
- Es crítico para la seguridad cuando este entero se usa como control de un bucle o para determinar el tamaño de la reserva de memoria, copia o concatenación de datos
- Se puede evitar:
 - Utilizar librerías que implementen "operaciones enteras seguras"
 - SafeInt (C++) o IntegerLib (C or C++)
 - Validación de entrada de los números introducidos para asegurar que están dentro del rango esperado

INTEGER OVERFLOW

- En el ejemplo el desbordamiento de entero conducir al bucle a un comportamiento indefinido
- Es posible que bytesRec desborde, creando continuamente un número menor que MAXGET y sobrescribiendo los primeros MAXGET-1 bytes de buf

```
short int bytesRec = 0;
char buf[SOMEBIGNUM];

while(bytesRec < MAXGET) {
    bytesRec += getFromInput(buf+bytesRec);
}
```


COMPROBACIÓN DE CADENAS

- Algunas funciones de C son problemáticas
 - Permiten el acceso a direcciones bajas de memoria sin comprobar el tamaño de los datos de retorno, pudiendo provocar estados o condiciones no deseadas
- Ejemplos:

NO USAR	USAR
<pre>void main (){ char buf[30]; gets(buf); }</pre>	<pre>void main (){ char buf[30]; fgets(buf,30,stdin); }</pre>

NO USAR

```
void main (){
    char buf[4];
    char sec[6]="aaaaaa"
    strcpy(buf,sec);
}
```

USAR

```
void main (){
    char buf[4];
    char sec[6]="aaaaaa";
    strncpy(buf,sec,buf_size-1);
    buf[buf_size-1]='\0';
}
```

COMPROBACIÓN DE CADENAS

Función	Gravedad	Solución
gets	La más alta	fgets(buf,size,stdin)
strcpy, strcat	Muy alta	strncpy, strncat
sprintf, vsprintf	Muy alta	snprintf, vsnprintf
Familia scanf	Muy alta	Especificadores de precisión
realpath, syslog	Muy alta (dependiendo de la implementación)	Maxpathlen y chequeos manuales
getopt, getopt_long, getpass	Muy alta (dependiendo de la implementación)	Cadenas truncadas de entrada de tamaño razonable

COMPROBACIÓN DE CADENAS

Datos corruptos aceptados de la fuente

Datos no chequeados afectan a otros datos

Datos corruptos se usan en un operador o función

Malos usos:

`system ()`

`printf ()`

`malloc ()`

`strcpy ()`

Envíos a
RDBMS
SQL-
injection

Incluído en
HTML
Cross
Site
Scripting

Vulnerabilidad:

Inyección de comandos

Manipulación
formato de
cadenas

Integer/Buffer
overflow

Buffer
overflow

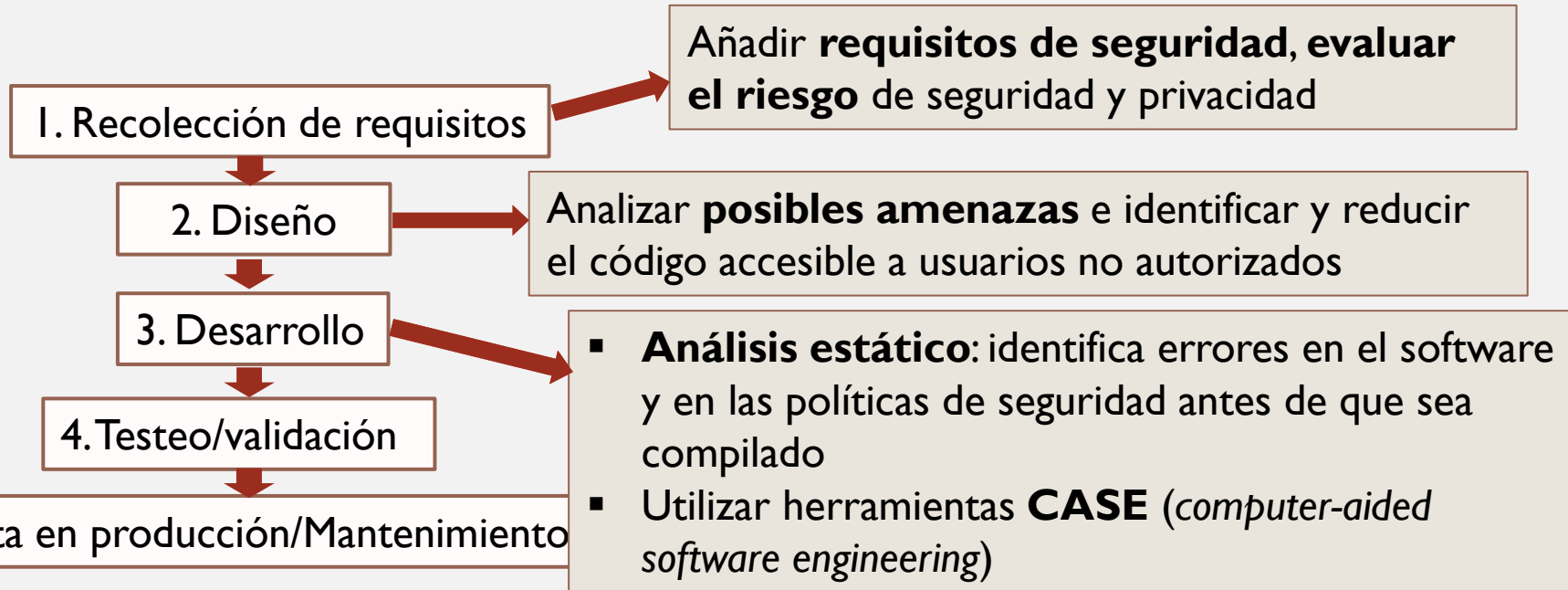
Fuente: Stanford University, Slides from: S. Bugrahe, A. Chou, I&T Dillig, D. Engler, J. Franklin, A. Aiken, ...

ERRORES SOFTWARE MÁS IMPORTANTES

Rank	ID	Name
[1]	CWE-787	Out-of-bounds Write
[2]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
[3]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
[4]	CWE-20	Improper Input Validation
[5]	CWE-125	Out-of-bounds Read
[6]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
[7]	CWE-416	User After Free
[8]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
[9]	CWE-352	Cross-Site Request Forgery (CSRF)
[10]	CWE-434	Unrestricted Upload of File with Dangerous Type
[11]	CWE-476	NULL Pointer Dereference
[12]	CWE-502	Deserialization of Untrusted Data
[13]	CWE-190	Integer Overflow or Wraparound
[14]	CWE-287	Improper Authentication
[15]	CWE-798	Use of Hard-coded Credentials
[16]	CWE-862	Missing Authorization
[17]	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
[18]	CWE-306	Missing Authentication for Critical Function
[19]	CWE-119	Improper Restriction of operations within the Bounds of a Memory Buffer
[20]	CWE-276	Incorrect Default Permissions
[21]	CWE-918	Server-side Request Forgery (SSRF)
[22]	CWE-362	Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition')
[23]	CWE-400	Uncontrolled Resource Consumption
[24]	CWE-611	Improper Restriction of XML External Entity Reference
[25]	CWE-94	Improper Control of Generation of Code ('Code Injection')

DESARROLLO DE SOFTWARE SEGURO

- Introducir seguridad en todas las fases del ciclo de desarrollo del software



DESARROLLO DE SOFTWARE SEGURO

- Introducir seguridad en todas las fases del ciclo de desarrollo del software

1. Recolección de requisitos

2. Diseño

3. Desarrollo

4. Testeo/validación

5. Puesta en producción/Mantenimiento

- **Fuzzing:** Descubre errores introduciendo datos aleatorios, malformados, o no esperados
- Entornos de **Pen-testing** para simular ataques
- **Testeo manual:** Analiza aspectos que requieren la intuición humana, como errores lógicos
- **Análisis dinámico:** Evalúa el programa mientras se está ejecutando

Descubrir **vulnerabilidades** con el software ya en uso y desarrollar **parches**

Bibliografía

CISSP - Certified Information Systems Security Professional-Exam Guide, Seventh Edition (Capítulos 3 y 8). Shon Harris y Fernando Maymí. 2016

CWE Common Weakness Enumeration, A Community-Developed List of Software Weakness Types (<http://cwe.mitre.org>)